

Node.js TypeScript API Boilerplate — Developer Guide

A production-ready **Express + TypeScript + MySQL** API framework with built-in authentication, RBAC, ORM, validation, logging, and rate-limiting. This guide walks you through creating new feature APIs end-to-end using the framework conventions.

Table of Contents

1. [Project Structure](#)
 2. [Getting Started](#)
 3. [Environment Configuration](#)
 4. [Framework Architecture Overview](#)
 5. [Creating a New CRUD API — Step-by-Step - Step 1 — Create the Migration - Step 2 — Create the Model - Step 3 — Create the Controller - Step 4 — Create the Routes - Step 5 — Register the Routes](#)
 6. [BaseModel API Reference](#)
 7. [Middleware Reference](#)
 8. [Utility Reference](#)
 9. [Error Handling Patterns](#)
 10. [Response Conventions](#)
 11. [Advanced Patterns](#)
-

1. Project Structure

```
src/
├─ config/           # Environment config loader
├─ controllers/      # Request handlers (one file per feature)
├─ core/
│   └─ BaseModel.ts  # Core ORM – extend this for every model
├─ db/
│   ├── connection.ts # MySQL connection pool
│   ├── migrate.ts    # Migration runner
│   ├── rollback.ts   # Migration rollback
│   ├── seed.ts       # Database seeder
│   └─ migrations/    # SQL migration files
├─ middleware/
│   ├── auth.middleware.ts    # JWT authentication
│   ├── error.middleware.ts   # Global error + 404 handler
│   ├── rbac.middleware.ts    # Role-based access control
│   ├── upload.middleware.ts  # Multer file upload
│   └─ validation.middleware.ts # Joi request validation
├─ models/           # Data models (extend BaseModel)
├─ routes/
│   ├── index.ts      # Root router – register all feature routes here
│   └─ *.routes.ts    # Feature-level routers
├─ services/         # Business logic / external integrations
├─ types/            # TypeScript interfaces & type declarations
├─ utils/
│   ├── apiError.ts    # ApiError factory class
│   ├── jwt.ts         # Token generation & verification
│   ├── logger.ts      # Winston logger
│   ├── password.ts    # bcrypt helpers
│   └─ response.ts     # Standard HTTP response helpers
└─ server.ts         # Express App class + entry point
```

2. Getting Started

Prerequisites

- Node.js $\geq$ 18.0.0
- npm $\geq$ 9.0.0
- MySQL 8.x

Installation

```
# Install dependencies
npm install

# Copy environment file
cp .env.example .env

# Edit .env with your database credentials and secrets

# Run migrations
npm run migrate

# Seed initial data (creates super admin + base roles/permissions)
npm run seed

# Start development server (hot reload)
npm run dev
```

Available Scripts

Script	Description
<code>npm run dev</code>	Start server with hot-reload (ts-node-dev)
<code>npm run build</code>	Compile TypeScript to <code>dist/</code>
<code>npm start</code>	Run compiled server
<code>npm run migrate</code>	Run pending migrations
<code>npm run migrate:rollback</code>	Roll back the last migration
<code>npm run seed</code>	Seed the database with initial data
<code>npm run typecheck</code>	Type-check without compiling
<code>npm test</code>	Run Jest tests

3. Environment Configuration

Copy `.env.example` to `.env` and fill in your values:

```
# Server
NODE_ENV=development
PORT=5000
API_VERSION=v1

# Database
DB_HOST=localhost
DB_PORT=3306
DB_NAME=base_db
DB_USER=root
DB_PASSWORD=your-password

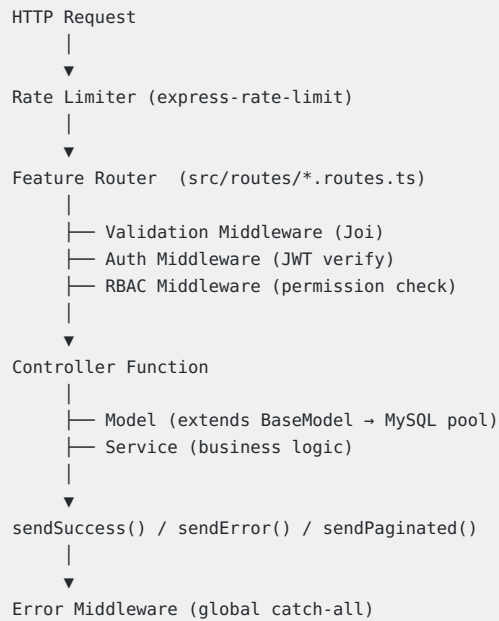
# JWT Secrets (use long random strings in production)
JWT_SECRET=your-super-secret-jwt-key
JWT_EXPIRES_IN=1h
JWT_REFRESH_SECRET=your-refresh-secret
JWT_REFRESH_EXPIRES_IN=7d

# CORS (comma-separated origins)
CORS_ORIGIN=http://localhost:3000,http://localhost:5173

# Rate Limiting (15 min window, 100 req max)
RATE_LIMIT_WINDOW_MS=900000
RATE_LIMIT_MAX_REQUESTS=100
```

All routes are prefixed with `/api/{API_VERSION}` — e.g., `/api/v1/auth/login`.

4. Framework Architecture Overview



Key design principles: - **Controllers** only handle HTTP concerns — parse request, call model/service, send response. - **Models** extend `BaseModel` and own all database access. - **Services** contain reusable business logic that doesn't belong in a model or controller. - All errors are passed to `next(error)` and handled centrally by `errorHandler`.

5. Creating a New CRUD API — Step-by-Step

Example Feature: Products API — full CRUD for a `products` table with category, soft delete, search, and pagination.

Step 1 — Create the Migration

Create a new SQL file in `src/db/migrations/`. Follow the naming convention `{NNN}_{feature_name}.sql`:

File: `src/db/migrations/002_products.sql`

```
-- =====
-- Migration: Products table
-- =====

CREATE TABLE IF NOT EXISTS products (
  id          INT UNSIGNED NOT NULL AUTO_INCREMENT,
  name        VARCHAR(200) NOT NULL,
  slug        VARCHAR(200) NOT NULL UNIQUE,
  description TEXT,
  price       DECIMAL(10, 2) NOT NULL DEFAULT 0.00,
  stock       INT NOT NULL DEFAULT 0,
  category    VARCHAR(100),
  is_active   TINYINT(1) NOT NULL DEFAULT 1,
  created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  deleted_at  DATETIME DEFAULT NULL,

  PRIMARY KEY (id),
  INDEX idx_slug (slug),
  INDEX idx_category (category),
  INDEX idx_is_active (is_active),
  INDEX idx_deleted_at (deleted_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Also create the rollback file:

File: `src/db/migrations/002_products.rollback.sql`

```
DROP TABLE IF EXISTS products;
```

Run the migration:

```
npm run migrate
```

Step 2 — Create the Model

Extend `BaseModel` in a new file under `src/models/`:

File: `src/models/product.model.ts`

```

import BaseModel from '../core/BaseModel';
import { DatabaseRecord, DbConnection } from '../types';

// Optional: define a typed interface for your record
export interface Product extends DatabaseRecord {
  id: number;
  name: string;
  slug: string;
  description: string | null;
  price: number;
  stock: number;
  category: string | null;
  is_active: boolean;
  created_at: Date;
  updated_at: Date;
  deleted_at: Date | null;
}

class ProductModel extends BaseModel {
  constructor() {
    super({
      tableName: 'products',

      // Only these fields can be mass-assigned (INSERT / UPDATE)
      fillable: ['name', 'slug', 'description', 'price', 'stock', 'category', 'is_active'],

      // These columns are removed from all query results
      hidden: [],

      // Fields searched by the search() method
      searchable: ['name', 'slug', 'description', 'category'],

      // Automatically cast column values when reading
      casts: {
        price: 'float',
        stock: 'int',
        is_active: 'boolean',
      },

      // Default values applied during create() if not provided
      defaults: {
        is_active: true,
        stock: 0,
      },

      // Auto-manage created_at / updated_at
      timestamps: true,

      // Use soft-delete (sets deleted_at instead of hard DELETE)
      softDelete: true,
    });
  }

  /**
   * Find a product by its unique slug.
   */
  async findBySlug(slug: string, transaction?: DbConnection): Promise<Product | null> {
    return this.findOne<Product>({ slug }, { transaction });
  }

  /**
   * Get all active products in a category with pagination.
   */
  async findByCategory(
    category: string,
    page = 1,
    limit = 20,
    transaction?: DbConnection
  ) {
    return this.paginate<Product>({
      where: { category, is_active: true },
    });
  }
}

```

```

        orderBy: 'name',
        direction: 'ASC',
        page,
        limit,
        transaction,
    });
}

/**
 * Lifecycle hook – runs before every INSERT.
 * Use this to auto-generate slugs, validate business rules, etc.
 */
protected async beforeCreate(data: DatabaseRecord): Promise<void> {
    if (!data.slug && data.name) {
        data.slug = (data.name as string)
            .toLowerCase()
            .replace(/^[^a-z0-9]+/g, '-')
            .replace(/^-|-$|_/g, '');
    }
}

/**
 * Decrement stock atomically – safe for concurrent requests.
 */
async decrementStock(id: number, qty: number, transaction?: DbConnection): Promise<void> {
    const sql = `
        UPDATE products
        SET stock = GREATEST(stock - ?, 0),
            updated_at = NOW()
        WHERE id = ? AND deleted_at IS NULL
    `;
    await this.raw(sql, [qty, id], { method: 'none', transaction });
}
}

export default new ProductModel();

```

ModelConfig Options

Option	Type	Default	Description
<code>tableName</code>	<code>string</code>	<i>required</i>	MySQL table name
<code>tableAlias</code>	<code>string</code>	first char of <code>tableName</code>	Alias used when building JOIN queries
<code>primaryKey</code>	<code>string</code>	<code>'id'</code>	Primary key column name
<code>fillable</code>	<code>string[]</code>	<code>[]</code>	Columns allowed in create/update
<code>guarded</code>	<code>string[]</code>	<code>[]</code>	Columns excluded from mass-assignment (alternative to <code>fillable</code>)
<code>hidden</code>	<code>string[]</code>	<code>[]</code>	Columns removed from all results
<code>timestamps</code>	<code>boolean</code>	<code>true</code>	Auto-manage <code>created_at</code> / <code>updated_at</code>
<code>softDelete</code>	<code>boolean</code>	<code>false</code>	Use <code>deleted_at</code> instead of hard DELETE
<code>searchable</code>	<code>string[]</code>	<code>[]</code>	Columns searched by <code>search()</code>
<code>casts</code>	<code>Record<string, CastType></code>	<code>{}</code>	Auto-cast values when reading
<code>defaults</code>	<code>Record<string, unknown></code>	<code>{}</code>	Default values for <code>create()</code>

Step 3 — Create the Controller

Controllers are thin HTTP-layer handlers. All database work is delegated to models.

File: `src/controllers/product.controller.ts`

```

import { Request, Response, NextFunction } from 'express';
import Joi from 'joi';
import productModel, { Product } from '../models/product.model';
import { ApiError } from '../utils/apiError';
import { sendSuccess, sendPaginated } from '../utils/response';
import { AuthRequest } from '../types';

// =====
// READ - List all products (paginated + optional search)
// =====
export const listProducts = async (req: Request, res: Response, next: NextFunction): Promise<void> => {
  try {
    const { page = 1, limit = 20, search, category } = req.query;

    // Use full-text search if a term is provided
    if (search) {
      const results = await productModel.search<Product>(search as string, {
        where: { is_active: true },
        limit: Number(limit),
      });
      sendSuccess(res, 'Products retrieved', results);
      return;
    }

    // Category filter with pagination
    if (category) {
      const result = await productModel.findByCategory(
        category as string,
        Number(page),
        Number(limit)
      );
      sendPaginated(res, result.data, result.pagination);
      return;
    }

    // Default: all products paginated
    const result = await productModel.paginate<Product>({
      where: { is_active: true },
      orderBy: 'created_at',
      direction: 'DESC',
      page: Number(page),
      limit: Number(limit),
    });

    sendPaginated(res, result.data, result.pagination, 'Products retrieved');
  } catch (error) {
    next(error);
  }
};

// =====
// READ - Get single product by ID
// =====
export const getProduct = async (req: Request, res: Response, next: NextFunction): Promise<void> => {
  try {
    const id = parseInt(req.params.id, 10);
    if (isNaN(id)) throw ApiError.badRequest('Invalid product ID');

    const product = await productModel.findById<Product>(id);
    if (!product) throw ApiError.notFound('Product not found');

    sendSuccess(res, 'Product retrieved', product);
  } catch (error) {
    next(error);
  }
};

// =====
// CREATE - Add a new product
// =====
export const createProduct = async (req: Request, res: Response, next: NextFunction): Promise<void> => {

```

```

    try {
      const { name, description, price, stock, category } = req.body;

      // Check slug uniqueness (slug is auto-generated in beforeCreate hook)
      const slug = name.toLowerCase().replace(/[^\a-z0-9]+/g, '-').replace(/^-|-$/, '');
      const existing = await productModel.findBySlug(slug);
      if (existing) throw ApiError.badRequest('A product with this name already exists');

      const product = await productModel.create<Product>({
        name,
        description,
        price,
        stock,
        category,
        // slug is auto-generated by the beforeCreate() lifecycle hook
      });

      sendSuccess(res, 'Product created', product, 201);
    } catch (error) {
      next(error);
    }
  };

  // =====
  // UPDATE – Edit an existing product
  // =====
  export const updateProduct = async (req: Request, res: Response, next: NextFunction): Promise<void> => {
    try {
      const id = parseInt(req.params.id, 10);
      if (isNaN(id)) throw ApiError.badRequest('Invalid product ID');

      const existing = await productModel.findById<Product>(id);
      if (!existing) throw ApiError.notFound('Product not found');

      const { name, description, price, stock, category, is_active } = req.body;

      const updated = await productModel.update<Product>(id, {
        name,
        description,
        price,
        stock,
        category,
        is_active,
      });

      sendSuccess(res, 'Product updated', updated);
    } catch (error) {
      next(error);
    }
  };

  // =====
  // DELETE – Soft-delete a product
  // =====
  export const deleteProduct = async (req: Request, res: Response, next: NextFunction): Promise<void> => {
    try {
      const id = parseInt(req.params.id, 10);
      if (isNaN(id)) throw ApiError.badRequest('Invalid product ID');

      const existing = await productModel.findById<Product>(id);
      if (!existing) throw ApiError.notFound('Product not found');

      // softDelete: true in model means this sets deleted_at, not a hard DELETE
      await productModel.delete(id);

      sendSuccess(res, 'Product deleted');
    } catch (error) {
      next(error);
    }
  };

  // =====

```

```
// RESTORE – Recover a soft-deleted product
// =====
export const restoreProduct = async (req: Request, res: Response, next: NextFunction): Promise<void> => {
  try {
    const id = parseInt(req.params.id, 10);
    const product = await productModel.restore<Product>(id);
    sendSuccess(res, 'Product restored', product);
  } catch (error) {
    next(error);
  }
};
```

Step 4 — Create the Routes

Define Joi validation schemas and wire middleware → controller in the router file.

File: `src/routes/product.routes.ts`

```

import { Router } from 'express';
import Joi from 'joi';
import * as productController from '../controllers/product.controller';
import { authenticate } from '../middleware/auth.middleware';
import { authorize } from '../middleware/rbac.middleware';
import { validate, commonSchemas } from '../middleware/validation.middleware';

const router = Router();

// ---- Validation schemas ----

const createProductBody = Joi.object({
  name:      Joi.string().min(2).max(200).required(),
  description: Joi.string().max(5000).optional().allow('', null),
  price:      Joi.number().positive().precision(2).required(),
  stock:      Joi.number().integer().min(0).default(0),
  category:   Joi.string().max(100).optional().allow(null),
});

const updateProductBody = Joi.object({
  name:      Joi.string().min(2).max(200),
  description: Joi.string().max(5000).allow('', null),
  price:      Joi.number().positive().precision(2),
  stock:      Joi.number().integer().min(0),
  category:   Joi.string().max(100).allow(null),
  is_active:  Joi.boolean(),
}).min(1); // At least one field must be provided

const listProductsQuery = Joi.object({
  page:      Joi.number().integer().min(1).default(1),
  limit:      Joi.number().integer().min(1).max(100).default(20),
  search:     Joi.string().max(200).optional(),
  category:  Joi.string().max(100).optional(),
});

const idParam = Joi.object({
  id: Joi.number().integer().positive().required(),
});

// ---- Public routes (no authentication required) ----

router.get(
  '/',
  validate({ query: listProductsQuery }),
  productController.listProducts
);

router.get(
 ('/:id',
  validate({ params: idParam }),
  productController.getProduct
);

// ---- Protected routes (authentication + RBAC required) ----

router.use(authenticate); // All routes below require a valid JWT

router.post(
  '/',
  validate({ body: createProductBody }),
  authorize('products', 'create'), // Check RBAC permission
  productController.createProduct
);

router.put(
 ('/:id',
  validate({ params: idParam, body: updateProductBody }),
  authorize('products', 'update'),
  productController.updateProduct
);

```

```

router.delete(
 ('/:id',
    validate({ params: idParam }),
    authorize('products', 'delete'),
    productController.deleteProduct
  );

router.post(
 ('/:id/restore',
    validate({ params: idParam }),
    authorize('products', 'delete'),
    productController.restoreProduct
  );

export default router;

```

Step 5 — Register the Routes

Add the feature router to the root router:

File: `src/routes/index.ts`

```

import { Router } from 'express';
import config from '../config';
import authRoutes from './auth.routes';
import rbacRoutes from './rbac.routes';
import productRoutes from './product.routes'; // ← ADD THIS

const router = Router();

router.use('/auth', authRoutes);
router.use('/rbac', rbacRoutes);
router.use('/products', productRoutes); // ← ADD THIS

router.get('/health', (_req, res) => {
  res.json({ status: 'UP', version: config.server.apiVersion, timestamp: new Date().toISOString() });
});

export default router;

```

Your endpoints are now live:

Method	Endpoint	Auth	Permission
GET	<code>/api/v1/products</code>	No	—
GET	<code>/api/v1/products/:id</code>	No	—
POST	<code>/api/v1/products</code>	Yes	<code>products:create</code>
PUT	<code>/api/v1/products/:id</code>	Yes	<code>products:update</code>
DELETE	<code>/api/v1/products/:id</code>	Yes	<code>products:delete</code>
POST	<code>/api/v1/products/:id/restore</code>	Yes	<code>products:delete</code>

6. BaseModel API Reference

All models inherit these methods. Generic type `T` defaults to `DatabaseRecord`.

Read

```
// Find all rows matching filters
await model.findAll<T>({
  where:    { is_active: true },           // WHERE clause
  columns:  ['id', 'name', 'price'],       // SELECT columns ('*' = all)
  joins:    [{ type: 'LEFT', table: 'categories', alias: 'c', on: 'products.category_id = c.id' }],
  orderBy:  'created_at',
  direction: 'DESC',
  groupBy:  'category',
  having:    'COUNT(*) > 5',
  limit:     20,
  offset:    40,
  transaction,
});

// Find by primary key
await model.findById<T>(id, { columns, joins, transaction });

// Find first row matching filters
await model.findOne<T>({ email: 'a@b.com' }, { columns, joins, transaction });

// Paginated result set
await model.paginate<T>({ page: 2, limit: 10, where: {...}, orderBy: 'name' });
// Returns: { data: T[], pagination: { page, limit, total, totalPages, hasMore } }

// Full-text search across searchable columns
await model.search<T>('coffee', { where: { is_active: true }, limit: 10 });

// Row count
await model.count({ is_active: true });

// Existence check
await model.exists({ email: 'taken@example.com' }); // boolean
```

Write

```
// Create one row (respects fillable, triggers beforeCreate/afterCreate)
await model.create<T>({ name: 'Widget', price: 9.99 });

// Create many rows (wrapped in a transaction automatically)
await model.createMany<T>([{ name: 'A' }, { name: 'B' }]);

// Update by primary key (respects fillable, triggers beforeUpdate/afterUpdate)
await model.update<T>(id, { price: 14.99 });

// Update multiple rows matching a WHERE filter
await model.updateWhere<T>({ category: 'old' }, { category: 'new' });
```

Delete & Restore

```
// Soft-delete (if softDelete: true) or hard DELETE
await model.delete(id);
await model.delete(id, { force: true }); // Always hard DELETE

// Delete by filter – returns number of affected rows
await model.deleteWhere({ is_active: false });

// Restore a soft-deleted row
await model.restore<T>(id);
```

Transactions

```
// Run multiple operations atomically
await model.transaction(async (conn) => {
  const order = await orderModel.create({ ... }, { transaction: conn });
  await productModel.decrementStock(productId, qty, conn);
  await paymentModel.create({ order_id: order.id, ... }, { transaction: conn });
  // Auto-commits on success, auto-rolls back on error
});
```

Raw SQL

```
// Run arbitrary SQL – useful for complex JOINS or aggregations
const result = await model.raw<MyType[]>(`
  SELECT p.*, c.name AS category_name
  FROM products p
  JOIN categories c ON p.category_id = c.id
  WHERE p.price BETWEEN ? AND `,
  [minPrice, maxPrice],
  { method: 'any' } // 'any' | 'one' | 'oneOrNone' | 'many' | 'none' | 'result'
);
```

WHERE Filter Operators

```
// Equality (default)
where: { status: 'active' }

// NULL check
where: { deleted_at: null }

// IN clause (pass an array)
where: { status: ['pending', 'processing'] }

// Custom operator
where: {
  price: { operator: '>=', value: 100 },
  stock: { operator: '<', value: 10 },
}
```

Lifecycle Hooks

Override these in your model subclass:

```

class ProductModel extends BaseModel {
  // Runs BEFORE INSERT – mutate `data` directly to modify what gets saved
  protected async beforeCreate(data: DatabaseRecord): Promise<void> {
    data.slug = generateSlug(data.name as string);
  }

  // Runs AFTER INSERT – receives the newly inserted row
  protected async afterCreate(result: DatabaseRecord): Promise<void> {
    await notifySlackChannel(`New product: ${result.name}`);
  }

  // Runs BEFORE UPDATE
  protected async beforeUpdate(id: number, data: DatabaseRecord): Promise<void> {
    console.log(`Updating product #${id}`);
  }

  // Runs AFTER UPDATE – receives the updated row
  protected async afterUpdate(result: DatabaseRecord): Promise<void> {}

  // Runs BEFORE DELETE
  protected async beforeDelete(id: number): Promise<void> {}

  // Runs AFTER DELETE
  protected async afterDelete(id: number): Promise<void> {}
}

```

7. Middleware Reference

authenticate — JWT Auth

```

import { authenticate } from '../middleware/auth.middleware';

// Apply to all routes in a router
router.use(authenticate);

// Or selectively
router.get('/profile', authenticate, controller.getProfile);

```

After authentication, `req.user` is populated:

```

req.user = {
  userId: number,
  email: string,
  clientId: number | undefined,
  roleId: number,
  roleSlug: string,
  level: number, // higher = more authority (super_admin = 100)
}

```

authorize — RBAC Permission Check

Always used **after** `authenticate`:

```
import { authorize } from '../middleware/rbac.middleware';

router.get('/products', authenticate, authorize('products', 'read'), controller.list);
router.post('/products', authenticate, authorize('products', 'create'), controller.create);
router.put('/products/:id', authenticate, authorize('products', 'update'), controller.update);
router.delete('/products/:id', authenticate, authorize('products', 'delete'), controller.delete);
```

- Super admin users (`level >= 100`) bypass all permission checks.
- The permission check looks up the `permissions` table for the user's role + module + action.
- A `scope` and `conditions` object are attached to `req.permission` for optional ABAC logic in the controller.

validate — Joi Request Validation

```
import { validate, commonSchemas } from '../middleware/validation.middleware';
import Joi from 'joi';

const bodySchema = Joi.object({
  email: commonSchemas.email.required(),
  password: commonSchemas.password.required(),
  name: Joi.string().min(2).max(100).required(),
});

const querySchema = commonSchemas.paginationQuery; // { page, limit }

const paramsSchema = Joi.object({
  id: Joi.number().integer().positive().required(),
});

router.post('/register', validate({ body: bodySchema }), controller.register);
router.get('/', validate({ query: querySchema }), controller.list);
router.get('/:id', validate({ params: paramsSchema }), controller.getOne);

// Validate multiple parts at once
router.put('/:id', validate({
  params: paramsSchema,
  body: updateSchema,
}), controller.update);
```

Common schemas available in `commonSchemas` :

Key	Description
<code>email</code>	Valid email string
<code>password</code>	Min 8 chars string
<code>phone</code>	Phone number pattern
<code>uuidParam</code>	<code>{ id: uuid }</code> object
<code>paginationQuery</code>	<code>{ page, limit }</code>
<code>dateRangeQuery</code>	<code>{ startDate, endDate }</code>
<code>latitude</code> / <code>longitude</code>	Geo coordinate numbers

asyncHandler — Promise Error Wrapper

Use this instead of manual `try/catch` if preferred:

```
import { asyncHandler } from '../middleware/error.middleware';

router.get('/:id', asyncHandler(async (req, res) => {
  const product = await productModel.findById(Number(req.params.id));
  if (!product) throw ApiError.notFound('Product not found');
  sendSuccess(res, 'Product found', product);
}));
```

8. Utility Reference

ApiError — HTTP Error Factory

```
import { ApiError } from '../utils/apiError';

throw ApiError.badRequest('Validation failed'); // 400
throw ApiError.unauthorized('Token expired'); // 401
throw ApiError.forbidden('Insufficient rights'); // 403
throw ApiError.notFound('Record not found'); // 404
throw ApiError.internal('Something broke'); // 500

// With extra error details
throw ApiError.badRequest('Validation failed', ['email is required', 'name too short']);
```

Response Helpers

```
import { sendSuccess, sendError, sendPaginated } from '../utils/response';

// Success — 200 by default
sendSuccess(res, 'Message', data);
sendSuccess(res, 'Created', newRecord, 201);

// Paginated list
sendPaginated(res, items, paginationMeta, 'Products retrieved');

// Error (rarely used directly — prefer throwing ApiError)
sendError(res, 'Something failed', 500);
```

Standard response shapes:

```
// sendSuccess
{ "success": true, "message": "...", "data": { ... } }

// sendPaginated
{
  "success": true,
  "message": "...",
  "data": [...],
  "pagination": { "page": 1, "limit": 20, "total": 150, "totalPages": 8, "hasMore": true }
}

// Error
{ "success": false, "message": "...", "errors": [...] }
```

Logger

```
import logger from '../utils/logger';

logger.info('Server started');
logger.warn('Deprecated usage detected');
logger.error('DB connection failed', error);
logger.debug('Query executed', { sql, params });
```

Logs are written to `./logs/` and also streamed to the console.

9. Error Handling Patterns

The global `errorHandler` middleware in `src/middleware/error.middleware.ts` is the last middleware registered in `server.ts`. It catches any error passed to `next(error)`.

Always use `next(error)` in catch blocks — never `res.send()` directly from catch:

```
export const myHandler = async (req: Request, res: Response, next: NextFunction) => {
  try {
    // ... logic
  } catch (error) {
    next(error); // ← delegates to errorHandler
  }
};
```

Throwing `ApiError` sends a clean, structured response:

```
// Results in: HTTP 404 { success: false, message: "Product not found" }
throw ApiError.notFound('Product not found');
```

Unhandled errors (non-`ApiError`) result in a generic HTTP 500 and are logged with full stack trace. In `development` mode, the stack trace is also included in the response body.

10. Response Conventions

Scenario	Status	Body
Successful GET	200	<code>{ success, message, data }</code>
Successful CREATE	201	<code>{ success, message, data }</code>
Successful DELETE	200	<code>{ success, message, data: null }</code>
Paginated list	200	<code>{ success, message, data[], pagination }</code>
Validation error	400	<code>{ success, message, errors[] }</code>
Unauthenticated	401	<code>{ success, message }</code>
Forbidden	403	<code>{ success, message }</code>
Not found	404	<code>{ success, message }</code>
Server error	500	<code>{ success, message }</code>

11. Advanced Patterns

JOIN Queries via findAll

```
const products = await productModel.findAll<Product>({
  columns: ['p.id', 'p.name', 'p.price', 'c.name AS category_name'],
  joins: [{
    type: 'LEFT',
    table: 'categories',
    alias: 'c',
    on: 'p.category_id = c.id',
  }],
  where: { 'p.is_active': true },
  orderBy: 'p.name',
});
```

Note: When using joins, the model auto-assigns a table alias (`p` for `products` by default — the first character of the table name). You can override it via `tableAlias` in the model config.

Multi-step Transactions

```
export const placeOrder = async (req: Request, res: Response, next: NextFunction) => {
  try {
    const order = await orderModel.transaction(async (conn) => {
      // All operations share the same connection/transaction
      const newOrder = await orderModel.create({
        user_id: req.user.userId,
        total: req.body.total,
      }, { transaction: conn });

      for (const item of req.body.items) {
        await orderItemModel.create({
          order_id: newOrder.id,
          product_id: item.productId,
          qty: item.qty,
          price: item.price,
        }, { transaction: conn });

        await productModel.decrementStock(item.productId, item.qty, conn);
      }

      return newOrder;
    }); // auto-commits here; auto-rolls back on any thrown error

    sendSuccess(res, 'Order placed', order, 201);
  } catch (error) {
    next(error);
  }
};
```

Soft Delete Behaviour

When `softDelete: true` is set in a model:

- `findAll()`, `findById()`, `findOne()`, `paginate()`, `search()` automatically exclude rows where `deleted_at IS NOT NULL`.
- `delete(id)` sets `deleted_at = NOW()` instead of running a `DELETE` statement.
- `delete(id, { force: true })` permanently deletes even when soft-delete is enabled.
- `restore(id)` sets `deleted_at = NULL`.
- `deleteWhere()` also soft-deletes matching rows.

Adding a Service Layer

For complex business logic, create a service file in `src/services/` :

```
// src/services/product.service.ts
import productModel, { Product } from '../models/product.model';
import { ApiError } from '../utils/apiError';

export const transferStock = async (fromId: number, toId: number, qty: number): Promise<void> => {
  await productModel.transaction(async (conn) => {
    const from = await productModel.findById<Product>(fromId, { transaction: conn });
    if (!from) throw ApiError.notFound('Source product not found');
    if ((from.stock as number) < qty) throw ApiError.badRequest('Insufficient stock');

    await productModel.raw(
      `UPDATE products SET stock = stock - ? WHERE id = ?`,
      [qty, fromId],
      { method: 'none', transaction: conn }
    );
    await productModel.raw(
      `UPDATE products SET stock = stock + ? WHERE id = ?`,
      [qty, toId],
      { method: 'none', transaction: conn }
    );
  });
};
```

Then call from the controller:

```
import * as productService from '../services/product.service';

export const transferStock = async (req: Request, res: Response, next: NextFunction) => {
  try {
    await productService.transferStock(
      Number(req.body.fromId),
      Number(req.body.toId),
      Number(req.body.qty)
    );
    sendSuccess(res, 'Stock transferred');
  } catch (error) {
    next(error);
  }
};
```

File Upload

The framework includes `multer` for file handling:

```
import { Router } from 'express';
import { uploadSingle } from '../middleware/upload.middleware';
import * as productController from '../controllers/product.controller';
import { authenticate } from '../middleware/auth.middleware';

const router = Router();

router.post(
  '/:id/image',
  authenticate,
  uploadSingle('image'), // field name in the multipart form
  productController.uploadImage
);
```

In the controller:

```
export const uploadImage = async (req: Request, res: Response, next: NextFunction) => {
  try {
    if (!req.file) throw ApiError.badRequest('No file uploaded');
    const filePath = req.file.path;

    await productModel.update(Number(req.params.id), { image_url: filePath });
    sendSuccess(res, 'Image uploaded', { path: filePath });
  } catch (error) {
    next(error);
  }
};
```

Quick Reference Checklist

When adding a new feature, work through this list in order:

- [] Create SQL migration in `src/db/migrations/{NNN}_{feature}.sql`
- [] Create rollback file `{NNN}_{feature}.rollback.sql`
- [] Run `npm run migrate`
- [] Create `src/models/{feature}.model.ts` extending `BaseModel`
- [] Create `src/controllers/{feature}.controller.ts`
- [] Create `src/routes/{feature}.routes.ts` with Joi validation + middleware
- [] Register the route in `src/routes/index.ts`
- [] (Optional) Add RBAC `module` + `action` rows in the DB for the new resource
- [] Test with your HTTP client (e.g., Postman, curl, Thunder Client)